

CS690S -AI Alignment - Fall 2026

Sagnik Chatterjee

September 24, 2026

1 Part 1

1.1 Question

Run the following code again

```
python test_gym.py
```

What do you notice happening? What specifically about MountainCar makes the problem difficult for RL?

1.2 Solution

The random policy jitters the car back and forth near the bottom of the valley and essentially never leaves it: every one of the 200 steps returns -1 and the episode ends with a return of -200.0 , with the car still in the basin.

MountainCar is hard for RL because the reward signal carries no information about progress. The engine is weaker than gravity, so the only way to reach the flag is to first drive away from it and pump energy into the system over many steps (a long, precisely correlated sequence of actions). A uniformly random policy has essentially zero probability of producing such a sequence within 200 steps, so the agent almost never sees a non -200 return. This means:

- Exploration / sparse-in-practice reward: all trajectories look equally bad (-200), so the reward gives no gradient toward the goal. The agent is stuck in a flat region of the return landscape, i.e. a local optimum where nothing it tries locally looks better than anything else.
- Long credit-assignment horizon: the action that matters (backing up the left hill) is rewarded only tens of steps later, when the car finally clears the right hill.
- Counter-intuitive optimal behavior: a greedy/myopic policy that pushes toward the goal never succeeds; the optimal policy must temporarily move in the wrong direction.

2 Part 2

2.1 Question

You will now learn how to solve the MountainCar task by driving the car yourself. Run

python mountain_car_play.py

Use the arrow keys to control the car by accelerating left and right. Note that if you run out of time you get a reward of -200.0 and the car will reset to the bottom of the hill. If you get to the flag in less than 200 steps you can get a higher score. Once you reach the flag the environment will restart at the bottom of the hill.

Keep practicing until you can reliably get out of the valley. You can see your score for each episode output on the command line. Experiment with different strategies: e.g.

1. Going right, then left, then right all the way up the hill
2. Going left, then right all the way up the hill.
3. Left, right, left, right up the hill.

Which strategy do you like best? What is the best score you can get as a human demonstrator?

2.2 Solution

I found strategy 2 (go left first to build up potential energy, then accelerate right all the way up) the easiest and most reliable, because it requires only one direction switch and so is easy to time correctly. Strategy 1 also works but wastes the first few steps; strategy 3 (left, right, left, right) is the hardest to time: if the switches are not in phase with the car's natural oscillation, the pumping cancels out and the episode times out at -200 .

Over roughly 19 practice episodes my returns ranged from -200.0 (timeouts, early on) to a best score of **-87.0** , with typical good demonstrations landing in the -140 to -160 range. The -87.0 episode used the "left then right" strategy with a single well-timed switch at the leftmost point of the swing. Table 1 lists every scored episode.

Table 1: Returns for my 18 scored practice episodes in mountain_car_play.py, in the order they were played.

episode	1	2	3	4	5	6	7	8	9
return	-165	-155	-88	-200	-176	-146	-169	-151	-157
episode	10	11	12	13	14	15	16	17	18
return	-138	-142	-87	-160	-159	-160	-200	-160	-147

3 Part 3

3.1 Question

Now we will teach a simple behavioral cloning (BC) agent to drive itself out of the valley.

First take a look at mountain_car_bc.py and try to get a basic understanding of what is happening. By default this code will collect a single demonstration, then parse the states and actions into tensors for easy input to a PyTorch neural network. It then trains a policy to imitate the demonstrations and evaluates the policy by testing it on different initial states. The command line will output the average, min, and max total rewards.

Try it out by running

```
python mountain_car_bc.py
```

Try to provide a good demonstration. Then watch what the agent has learned. Does it do a good job imitating? Does it ever get stuck? What is the average, min, and max?

3.2 Solution

The demonstration I gave had return -151.0 . BC training drove the cross-entropy loss down smoothly from 1.07 to 0.29 over 100 full-batch iterations, so the policy does fit the demonstration data well.

Behaviorally the imitation is only partly successful. The learned policy reproduces the general left-then-right pumping motion, but it is brittle: from initial states that differ from the one state the demonstration started in, it drifts off the demonstrated trajectory and then does not know what to do, and in 2 of 6 evaluation episodes it got stuck oscillating in the valley and timed out. Table 2 shows the training loss and Table 3 the per-episode evaluation returns.

Table 2: Part 3 BC training loss (1 demonstration, 100 full-batch iterations), sampled every 10 iterations.

iteration	0	10	20	30	40	50	60	70	80	90	99
BC loss	1.0681	0.8634	0.8158	0.7486	0.6565	0.5703	0.4791	0.4058	0.3511	0.3107	0.2852

Table 3: Part 3 evaluation returns over 6 episodes. Average -177.5 , minimum -200.0 , maximum -146.0 .

evaluation episode	0	1	2	3	4	5
return	-200	-184	-200	-149	-146	-186

So its best episodes are about as good as my demonstration (-146 vs. -151), but it fails outright a third of the time. This is the classic compounding error / covariate shift problem of BC: a single demonstration covers a very narrow slice of the state space, and once a small prediction error pushes the car into a state that never appeared in the data, the policy's predictions there are essentially unconstrained.

4 Part 4

4.1 Question

Let's give more than one demonstration. Run the following to give 5 good demonstrations. If you mess up during one demo, feel free to restart until you give 5 good demos and try to keep to a consistent strategy.

```
python mountain_car_bc.py --num_demos 5
```

Report the average min and max returns. Did it perform any better? Why or why not? Does the agent copy the strategy you used?

4.2 Solution

With 5 demonstrations (returns $-192, -160, -169, -162, -92$) the learned policy is clearly better and, importantly, much more consistent.

Table 4: Part 3 versus Part 4 evaluation statistics over 6 episodes.

	1 demo (Part 3)	5 demos (Part 4)
average return	-177.5	-145.8
minimum return	-200.0	-153.0
maximum return	-146.0	-141.0

Table 5: Part 4 evaluation returns over 6 episodes, all of which reach the flag.

evaluation episode	0	1	2	3	4	5
return	-144	-144	-143	-150	-153	-141

Table 6: Part 4 BC training loss (5 demonstrations, 100 full-batch iterations), sampled every 10 iterations.

iteration	0	10	20	30	40	50	60	70	80	90	99
BC loss	1.1298	0.9885	0.9562	0.9083	0.8679	0.7754	0.6963	0.6235	0.5703	0.5398	0.5230

Every one of the 6 evaluation episodes now reaches the flag (no -200 timeouts), and the spread of returns collapsed from 54 to 12.

Why it is better: more demonstrations cover more of the state space, in particular more distinct positions and velocities along the pumping swing and more distinct initial states, so the policy is asked to extrapolate much less at test time and small errors no longer push it into unseen states. The average return also beats the average of my own demonstrations (-155), because the network averages over my inconsistent timing and produces a smoother, more canonical version of the strategy.

Note that the final BC loss is higher than in Part 3 (0.52 vs. 0.29): with 5 demos there is more conflicting data (I did not switch direction at exactly the same state every time), so the fit is worse but the generalization is better. The agent does copy my strategy: it plays the left-then-right single-switch policy I demonstrated, not some other solution, which is expected, since BC has no reward signal and can only reproduce the demonstrated behavior mode.

5 Part 5

5.1 Question

What do you think will happen if we give good and bad demonstrations? You will now give two demonstrations. For the first one, just press the right arrow key for the entire episode until it restarts. Then for the second demo, give a good demonstration that quickly gets out of the valley.

```
python mountain_car_bc.py --num_demos 2
```

What does the policy learn? Why might bad demonstrations be a problem? Briefly suggest one potential idea for making BC robust to bad demonstrations, as long as they are a minority of demonstrations.

5.2 Solution

I expected the bad demonstration to pull the policy toward always accelerating right, and that is essentially what happened, but the failure was even more complete than that. With one all-right demonstration (-200.0) and one good demonstration (-200.0 as recorded for that episode), the learned policy never escapes the valley. All six evaluation episodes returned -200.0 , so the average, minimum and maximum returns are all -200.0 .

Table 7: Part 5 BC training loss (1 bad plus 1 good demonstration), sampled every 10 iterations. The loss plateaus near 0.59 and stops improving.

iteration	0	10	20	30	40	50	60	70	80	90	99
BC loss	1.2432	0.9600	0.7796	0.6149	0.5974	0.5985	0.5966	0.5936	0.5924	0.5921	0.5917

The BC loss also plateaus at ≈ 0.59 after about 40 iterations and stops improving, which is the signature of irreducible label noise: the two demonstrations visit overlapping states but label them with different actions, so no deterministic policy can fit both. Cross-entropy responds by predicting something close to the state-conditional average of the two behaviors, and because the argmax of that average is dominated by the “push right” action (which appears in both the bad demo and parts of the good one), the policy just pushes right and never builds momentum. Averaging two valid-but-different behaviors gives behavior that is worse than either.

Why bad demonstrations are a problem. BC is pure supervised learning on (s, a) pairs and has no notion of demonstration quality: every transition contributes equally to the loss, so a bad demonstrator is imitated just as faithfully as a good one. Worse, in MountainCar the good and bad policies disagree precisely in the states that matter (when to switch direction), so the conflict is not averaged away harmlessly, and it destroys the one behavior that solves the task.

One idea for robustness. Weight demonstrations rather than treating them uniformly: since we can evaluate each demonstration’s return in the environment (or its episode length / whether it reached the flag), fit BC with per-demonstration weights $w_i \propto \exp(R_i/\beta)$, or simply filter to the top- k / above-median demonstrations before training. Because bad demos are assumed to be a minority, the median return is a good threshold and the outliers get (near-)zero weight. If no reward is available at all, an unsupervised variant works similarly: fit BC, then compute each transition’s loss under the fitted policy and down-weight or drop the demonstrations with the highest disagreement (a trimmed-loss / RANSAC-style approach), iterating a few times so that the majority mode wins and the inconsistent minority is discarded.

6 Part 6

6.1 Question

At a conceptual level, describe what changes you would need to make to the BC code to implement BCO(0), as described in the paper we read in class: <https://arxiv.org/pdf/1805.01954.pdf>. Answer this question before starting Part 7.

6.2 Solution

BCO(0) removes the assumption that the demonstrations contain actions: we observe only the state sequences $\{(s_t, s_{t+1})\}$ of the demonstrator and must infer the actions ourselves before we can run BC. Conceptually there are three changes to the BC code, and one full pass through them ($\alpha = 0$ means no post-demonstration iteration):

1. Pre-demonstration / self-supervised data collection. Before touching the demonstrations, interact with the environment using a random policy and record (s_t, a_t, s_{t+1}) transitions. Here the actions are known because we chose them, so this data is free and needs no human. This is what `collect_random_interaction_data` provides.
2. Learn an inverse dynamics model. Train a new network $M_\theta(a \mid s_t, s_{t+1})$ on the random interaction data. Since the action space is discrete (3 actions), this is again a classification problem trained with cross-entropy, structurally the same training loop as BC, but with input (s_t, s_{t+1}) (i.e. 4 input features instead of 2) and the executed action as the label. This model is trained only on the self-generated data, never on the demonstrations.
3. Label the demonstrations, then clone. Discard the demonstrator's true actions (they are assumed unavailable) and instead run each demonstration transition (s_t, s_{t+1}) through the inverse dynamics model, taking $\hat{a}_t = \arg \max_a M_\theta(a \mid s_t, s_{t+1})$. Feed the resulting $(s_t, \hat{a}_t)$ dataset into the existing `train_policy` BC routine and evaluate as before.

So the policy network, the BC loss, and the evaluation loop stay exactly the same; what is added is a second network plus its own data-collection and training stage, and the action labels become predicted rather than ground truth. The key practical risk is a distribution mismatch: the inverse dynamics model is trained on states a random policy visits (near the bottom of the valley) but is applied to states the expert visits (high up on both hills), so its accuracy on the demonstration data can be much worse than its training accuracy. (BCO(α) fixes this by alternating: roll out the current learned policy, add those transitions to the inverse-dynamics training set, retrain, and re-label. With $\alpha = 0$ we do a single pass.)

7 Part 7

7.1 Question

Implement and test BCO(0) by modifying the starter code in `mountain_car_bco.py`. This starter code is identical to the BC code, except: (1) the function `collect_random_interaction_data` has been added to help with learning the inverse dynamics model; (2) a new argument for number of inverse dynamics training iterations has been

added; and (3) some comments have been added about where to use the inverse dynamics model instead of ground-truth actions. You may need to experiment with different neural network sizes and training iterations to get the inverse dynamics model to work well. Report how well it works and what you tried to get it to work, including stats on the accuracy of prediction when tested on the demonstration data (training should be done with the random interaction data, not the demonstrations).

7.2 Solution

Implementation. I implemented BCO(0) in `mountain_car_bco.py` exactly along the three stages above: an `InverseDynamicsNetwork` (input (s_t, s_{t+1}) , 3-way output) trained with full-batch Adam and cross-entropy on random interaction data, then used to relabel the demonstration transitions, which are then fed to the unchanged BC routine. The demonstrations are cached in `demos.pkl` so that every configuration is compared on the same 249 demonstration transitions (5 demos), and each run appends a row of config + metrics to a CSV. I ran a sweep of 96 runs (`run_p7.sh`, `results_p7.csv`) over hidden width, depth, learning rate, number of inverse dynamics iterations, number of BC iterations, amount of random data, input normalization, and action repeat, with 3 seeds per configuration.

What I had to do to make the inverse dynamics model work. Two changes mattered far more than network size:

1. Input representation / normalization. The raw features are on wildly different scales: position lives in $[-1.2, 0.6]$, while a single action changes velocity by only ± 0.001 . I therefore feed $(s_t, s_{t+1} - s_t)$ (position/velocity plus the delta) and standardize all 4 features using statistics computed from the random interaction data. Without this the action-identifying signal is numerically buried and training is much slower and less reliable.
2. State coverage of the random data (the real bottleneck). A uniformly random policy hardly leaves the bottom of the valley, so the model is trained on positions in roughly $[-0.73, -0.31]$ but applied to demonstration states spanning $[-0.94, 0.50]$. Adding an `-action_repeat` option, holding each (still uniformly random) action for k steps so the car builds momentum and covers far more of the track, extends the random-data range to roughly $[-1.2, 0.48]$ and essentially eliminates the mismatch.

Accuracy results. Training accuracy on the random interaction data is almost always 1.000, so it is not a useful diagnostic; the informative number is accuracy on the (held-out) demonstration transitions. Means over 3 seeds:

The best individual runs reached 100% demonstration-labelling accuracy and returns of -147 to -150 (with up to 6/6 successful evaluation episodes and a best single episode of -122), i.e. roughly comparable to the BC-with-true-actions result of Part 4 (-145.8), which is the natural upper bound for BCO(0) here.

Where the errors are. Breaking the baseline’s demonstration accuracy down confirms the coverage diagnosis:

- By action: `accelerate-left` and `no-op` are predicted at 100%, while `accelerate-right` drops to ≈ 0.82 , because the random policy rarely produces the sustained rightward motion the expert uses to climb.

Table 8: Inverse dynamics accuracy on the 249 demonstration transitions and resulting policy performance, averaged over 3 seeds per configuration.

configuration	demo acc.	avg. return	success rate
baseline (hidden 64, 2 layers, 5 random episodes, repeat 1)	0.869	-184.7	0.39
more random data (50 to 200 episodes, repeat 1)	0.974	-180.6	0.41
action repeat 2 to 16 (5 random episodes)	0.995	-175.4	0.48
action repeat 8 with 50 random episodes	0.997	-166.2	0.67

- By state region: on demonstration states inside the position range covered by the random data, accuracy is 1.00; on states outside it, accuracy falls to ≈ 0.79 . With action repeat, almost all demonstration states fall inside the covered range and accuracy goes to 1.00 in both buckets.

Things that did not help. Width and depth were largely irrelevant once normalization was in place. Hidden width 4 already gives ≈ 0.98 demonstration accuracy, and width 64 with 3 or 4 layers is no better (mean returns -189.5 for the width sweep, -192.3 for the depth sweep). Training the inverse dynamics model longer was mildly harmful (iters sweep: mean demo accuracy 0.789, mean return -195.0): it overfits the narrow random-data distribution and generalizes worse to expert states. Likewise, more BC iterations (500/1000 instead of 100) hurt (mean return -193.8), since the policy then fits the noisy predicted labels too tightly. Variance across seeds is large throughout, since the same configuration can go from 0% to 100% success, so all conclusions above are drawn from the multi-seed means rather than single runs.

Summary. BCO(0) works on MountainCar, but its performance is governed almost entirely by whether the self-supervised interaction data covers the states the demonstrator visits. Fixing the input representation and broadening random-policy coverage (action repeat + more episodes) takes demonstration-labelling accuracy from $\approx 87\%$ to $\approx 100\%$ and the success rate from 0.39 to 0.67, recovering close to the performance of BC with ground-truth actions, consistent with the paper’s motivation for the post-demonstration iterations of BCO(α), which address exactly this mismatch.